2
Tooling and Installation

This chapter presents all the essential tools that will be used throughout the book, especially in implementing and deploying the LLM Twin project. At this point in the book, we don't plan to present in-depth LLM, RAG, MLOps, or LLMOps concepts. We will quickly walk you through our tech stack and prerequisites to avoid repeating ourselves throughout the book on how to set up a particular tool and why we chose it. Starting with *Chapter 3*, we will begin exploring our LLM Twin use case by implementing a data collection ETL that crawls data from the internet.

In the first part of the chapter, we will present the tools within the Python ecosystem to manage multiple Python versions, create a virtual environment, and install the pinned dependencies required for our project to run. Alongside presenting these tools, we will also show how to install the `LLM-Engineers-Handbook` repository on your local machine (in case you want to try out the code yourself): https://github.com/Packt-Publishing/LLM-Engineers-Handbook.

Next, we will explore all the MLOps and LLMOps tools we will use, starting with more generic tools, such as a model registry, and moving on to more LLM-oriented tools, such as LLM evaluation and prompt monitoring tools. We will also understand how to manage a project with multiple ML pipelines using ZenML, an orchestrator bridging the gap between ML and MLOps. Also, we will quickly explore what databases we will use for NoSQL and vector storage. We will show you how to run all these components on your local machine using Docker. Lastly, we will quickly review

matically. We will also explore SageMaker and why we use it to train and deploy our open-source LLMs.

If you are familiar with these tools, you can safely skip this chapter. We also explain how to install the project and set up all the necessary components in the repository's `README`. Thus, you also have the option to use that as more concise documentation if you plan to run the code while reading the book.

To sum all that up, in this chapter, we will explore the following topics:

- Python ecosystem and project installation
- MLOps and LLMOps tooling
- Databases for storing unstructured and vector data
- Preparing for AWS

By the end of this chapter, you will be aware of all the tools we will use across the book. Also, you will have learned how to install the `LLM-Engineers-Handbook` repository, set up the rest of the tools, and use them if you run the code while reading the book.

Python ecosystem and project installation

Any Python project needs three fundamental tools: the Python interpreter, dependency management, and a task execution tool. The Python interpreter executes your Python project as expected. All the code within the book is tested with Python 3.11.8. You can download the Python interpreter from here: https://www.python.org/downloads/. We recommend installing the exact Python version (Python 3.11.8) to run the LLM Twin project using `pyenv`, making the installation process straightforward.

Instead of installing multiple global Python versions, we recommend managing them using `pyenv`, a Python version management tool that lets you manage multiple Python versions between projects. You can install it using this link: https://github.com/pyenv/pyenv?tab=readme-ov-file#installation.

After you have installed `pyenv`, you can install the latest version of Python 3.11, using `pyenv`, as follows:

```
pyenv install 3.11.8
```

Now list all installed Python versions to see that it was installed correctly:

```
pyenv versions
```

You should see something like this:

```
# * system
#   3.11.8
```

To make Python 3.11.8 the default version across your entire system (whenever you open a new terminal), use the following command:

```
pyenv global 3.11.8
```

However, we aim to use Python 3.11.8 locally only in our repository. To achieve that, first, we have to clone the repository and navigate to it:

```
git clone https://github.com/PacktPublishing/LLM-Engineers-Handbook.git
cd LLM-Engineers-Handbook
```

Because we defined a `.python-version` file within the repository, `pyenv` will know to pick up the version from that file and use it locally whenever you are working within that folder. To double-check that, run the following command while you are in the repository:

```
python --version
```

It should output:

```
# Python 3.11.8
```

To create the `.python-version` file, you must run `pyenv local 3.11.8` once. Then, `pyenv` will always know to use that Python version while working within a specific directory.

Now that we have installed the correct Python version using `pyenv`, let's move on to Poetry, which we will use as our dependency and virtual environment manager.

Poetry: dependency and virtual environment management

Poetry is one of the most popular dependency and virtual environment managers within the Python ecosystem. But let's start by clarifying what a dependency manager is. In Python, a dependency manager allows you to specify, install, update, and manage external libraries or packages (dependencies) that a project relies on. For example, this is a simple Poetry requirements file that uses Python 3.11 and the `requests` and `numpy` Python packages.

```
[tool.poetry.dependencies]
python = "^3.11"
requests = "^2.25.1"
numpy = "^1.19.5"
[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

 **Quick tip**: Enhance your coding experience with the **AI Code Explainer** and **Quick Copy** features. Open this book in the next-gen Packt Reader. Click the **Copy** button (**1**) to quickly copy code into your coding environment, or click the **Explain** button (**2**) to get the AI assistant to explain a block of code to you.

 **The next-gen Packt Reader** is included for free with the purchase of this book. Unlock it by scanning the QR code below or visiting https://www.packtpub.com/unlock/9781836200079.

By using Poetry to pin your dependencies, you always ensure that you install the correct version of the dependencies that your projects work with. Poetry, by default, saves all its requirements in `pyproject.toml` files, which are stored at the root of your repository, as you can see in the cloned LLM-Engineers-Handbook repository.

Another massive advantage of using Poetry is that it creates a new Python virtual environment in which it installs the specified Python version and requirements. A virtual environment allows you to isolate your project's dependencies from your global Python dependencies and other projects. By doing so, you ensure there are no version clashes between projects. For example, let's assume that Project A needs `numpy == 1.19.5`, and Project B needs `numpy == 1.26.0`. If you keep both projects in the glob-

al Python environment, that will not work, as Project B will override Project A's `numpy` installation, which will corrupt Project A and stop it from working. Using Poetry, you can isolate each project in its own Python environment with its own Python dependencies, avoiding any dependency clashes.

You can install Poetry from here: https://python-poetry.org/docs/. We use Poetry 1.8.3 throughout the book. Once Poetry is installed, navigate to your cloned LLM-Engineers-Handbook repository and run the following command to install all the necessary Python dependencies:

```
poetry install --without aws
```

This command knows to pick up all the dependencies from your repository that are listed in the `pyproject.toml` and `poetry.lock` files. After the installation, you can activate your Poetry environment by running `poetry shell` in your terminal or by prefixing all your CLI commands as follows: `poetry run <your command>`.

One final note on Poetry is that it locks down the exact versions of the dependency tree in the `poetry.lock` file based on the definitions added to the `project.toml` file. While the `pyproject.toml` file may specify version ranges (e.g., `requests = "^2.25.1"`), the `poetry.lock` file records the exact version (e.g., `requests = "2.25.1"`) that was installed. It also locks the versions of sub-dependencies (dependencies of your dependencies), which may not be explicitly listed in your `pyproject.toml` file. By locking all the dependencies and sub-dependencies to specific versions, the `poetry.lock` file ensures that all project installations use the same versions of each package. This consistency leads to predictable behavior, reducing the likelihood of encountering "works on my machine" issues.

Other tools similar to Poetry are Venv and Conda for creating virtual environments. Still, they lack the dependency management option. Thus, you must do it through Python's default `requirements.txt` files, which are less powerful than Poetry's `lock` files. Another option is Pipenv, which feature-wise is more like Poetry but slower, and `uv`, which is a replacement for Poetry built in Rust, making it blazing fast. `uv` has lots of potential to replace Poetry, making it worthwhile to test out: https://github.com/astral-sh/uv.

The final piece of the puzzle is to look at the task execution tool we used to manage all our CLI commands.

Poe the Poet: task execution tool

Poe the Poet is a plugin on top of Poetry that is used to manage and execute all the CLI commands required to interact with the project. It helps you define and run tasks within your Python project, simplifying automation and script execution. Other popular options are Makefile, Invoke, or shell scripts, but Poe the Poet eliminates the need to write separate shell scripts or Makefiles for managing project tasks, making it an elegant way to manage tasks using the same configuration file that Poetry already uses for dependencies.

When working with Poe the Poet, instead of having all your commands documented in a README file or other document, you can add them directly to your `pyproject.toml` file and execute them in the command line with an alias. For example, using Poe the Poet, we can define the following tasks in a `pyproject.toml` file:

```
[tool.poe.tasks]
test = "pytest"
format = "black ."
start = "python main.py"
```

You can then run these tasks using the `poe` command:

```
poetry poe test
poetry poe format
poetry poe start
```

You can install Poe the Poet as a Poetry plugin, as follows:

```
poetry self add 'poethepoet[poetry_plugin]'
```

To conclude, using a tool as a façade over all your CLI commands is necessary to run your application. It significantly simplifies the application's complexity and enhances collaboration as it acts as out-of-the-box documentation.

Assuming you have `pyenv` and Poetry installed, here are all the commands you need to run to clone the repository and install the dependencies and Poe the Poet as a Poetry plugin:

```
git clone https://github.com/PacktPublishing/LLM-Engineers-Handbook.gitcd LLM-Engi
poetry install --without aws
poetry self add 'poethepoet[poetry_plugin]'
```

To make the project fully operational, there are still a few steps to follow, such as filling out a `.env` file with your credentials and getting tokens from OpenAI and Hugging Face. But this book isn't an installation guide, so we've moved all these details into the repository's README as they are useful only if you plan to run the repository: https://github.com/PacktPublishing/LLM-Engineers-Handbook.

Now that we have installed our Python project, let's present the MLOps tools we will use in the book. If you are already familiar with these tools, you can safely skip the following tooling section and move on to the *Databases for storing unstructured and vector data* section.

MLOps and LLMOps tooling

This section will quickly present all the MLOps and LLMOps tools we will use throughout the book and their role in building ML systems using MLOps best practices. At this point in the book, we don't aim to detail all the MLOps components we will use to implement the LLM Twin use case, such as model registries and orchestrators, but only provide a quick idea of what they are and how to use them. As we develop the LLM Twin project throughout the book, you will see hands-on examples of how we

use all these tools. In *Chapter 11*, we will dive deeply into the theory of MLOps and LLMOps and connect all the dots. As the MLOps and LLMOps fields are highly practical, we will leave the theory of these aspects to the end, as it will be much easier to understand it after you go through the LLM Twin use case implementation.

Also, this section is not dedicated to showing you how to set up each tool. It focuses primarily on what each tool is used for and highlights the core features used throughout this book.

Still, using Docker, you can quickly run the whole infrastructure locally. If you want to run the steps within the book yourself, you can host the application locally with these three simple steps:

1. Have Docker 27.1.1 (or higher) installed.
2. Fill your `.env` file with all the necessary credentials as explained in the repository README.
3. Run `poetry poe local-infrastructure-up` to locally spin up ZenML (`http://127.0.0.1:8237/`) and the MongoDB and Qdrant databases.

You can read more details on how to run everything locally in the LLM-Engineers-Handbook repository README: https://github.com/PacktPublishing/LLM-Engineers-Handbook. Within the book, we will also show you how to deploy each component to the cloud.

Hugging Face: model registry

A model registry is a centralized repository that manages ML models throughout their lifecycle. It stores models along with their metadata, version history, and performance metrics, serving as a single source of truth. In MLOps, a model registry is crucial for tracking, sharing, and documenting model versions, facilitating team collaboration. Also, it is a fundamental element in the deployment process as it integrates with **continuous integration and continuous deployment (CI/CD)** pipelines.

We used Hugging Face as our model registry, as we can leverage its ecosystem to easily share our fine-tuned LLM Twin models with anyone who reads the book. Also, by following the Hugging Face model registry interface, we can easily integrate the model with all the frameworks around the LLMs ecosystem, such as Unsloth for fine-tuning and SageMaker for inference.

Our fine-tuned LLMs are available on Hugging Face at:

- **TwinLlama 3.1 8B** (after fine-tuning): https://huggingface.co/mlabonne/TwinLlama-3.1-8B
- **TwinLlama 3.1 8B DPO** (after preference alignment): https://huggingface.co/mlabonne/TwinLlama-3.1-8B-DPO

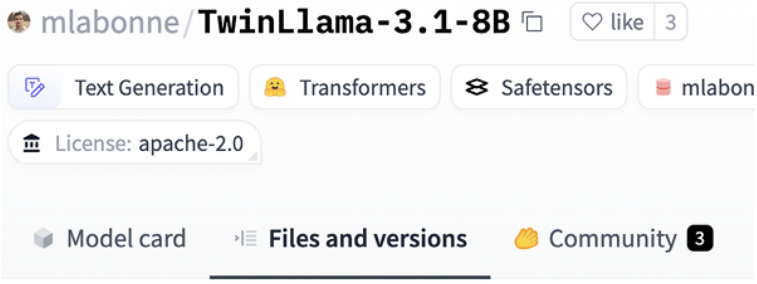

Figure 2.1: Hugging Face model registry example

For a quick demo, we have them available on Hugging Face Spaces:

- **TwinLlama 3.1 8B:** https://huggingface.co/spaces/mlabonne/TwinLlama-3.1-8B
- **TwinLlama 3.1 8B DPO:** https://huggingface.co/spaces/mlabonne/TwinLlama-3.1-8B-DPO

Most ML tools provide model registry features. For example, ZenML, Comet, and SageMaker, which we will present in future sections, also offer their own model registries. They are good options, but we picked Hugging Face solely because of its ecosystem, which provides easy shareability and integration throughout the open-source environment. Thus, you will usually select the model registry that integrates the most with your project's tooling and requirements.

ZenML: orchestrator, artifacts, and metadata

ZenML acts as the bridge between ML and MLOps. Thus, it offers multiple MLOps features that make your ML pipeline traceability, reproducibility, deployment, and maintainability easier. At its core, it is designed to create reproducible workflows in machine learning. It addresses the challenge of transitioning from exploratory research in Jupyter notebooks to a production-ready ML environment. It tackles production-based replication issues, such as versioning difficulties, reproducing experiments, organizing complex ML workflows, bridging the gap between training and deployment, and tracking metadata. Thus, ZenML's main features are orchestrating ML pipelines, storing and versioning ML pipelines as outputs, and attaching metadata to artifacts for better observability.

Instead of being another ML platform, ZenML introduced the concept of a *stack,* which allows you to run ZenML on multiple infrastructure options. A stack will enable you to connect ZenML to different cloud services, such as:

- An orchestrator and compute engine (for example, AWS SageMaker or Vertex AI)
- Remote storage (for instance, AWS S3 or Google Cloud Storage buckets)
- A container registry (for example, Docker Registry or AWS ECR)

Thus, ZenML acts as a glue that brings all your infrastructure and tools together in one place through its *stack* feature, allowing you to quickly iterate through your development processes and easily monitor your entire ML system. The beauty of this is that ZenML doesn't vendor-lock you into any cloud platform. It completely abstracts away the implementation of

your Python code from the infrastructure it runs on. For example, in our LLM Twin use case, we used the AWS stack:

- SageMaker as our orchestrator and compute
- S3 as our remote storage used to store and track artifacts
- ECR as our container registry

However, the Python code contains no S3 or ECR particularities, as ZenML takes care of them. Thus, we can easily switch to other providers, such as Google Cloud Storage or Azure. For more details on ZenML *stacks*, you can start here: https://docs.zenml.io/user-guide/production-guide/understand-stacks.

We will focus only on the ZenML features used throughout the book, such as orchestrating, artifacts, and metadata. For more details on ZenML, check out their starter guide: https://docs.zenml.io/user-guide/starter-guide.

The local version of the ZenML server comes installed as a Python package. Thus, when running `poetry install`, it installs a ZenML debugging server that you can use locally. In *Chapter 11*, we will show you how to use their cloud serverless option to deploy the ML pipelines to AWS.

Orchestrator

An orchestrator is a system that automates, schedules, and coordinates all your ML pipelines. It ensures that each pipeline—such as data ingestion, preprocessing, model training, and deployment—executes in the correct order and handles dependencies efficiently. By managing these processes, an orchestrator optimizes resource utilization, handles failures gracefully, and enhances scalability, making complex ML pipelines more reliable and easier to manage.

How does ZenML work as an orchestrator? It works with **pipelines** and **steps**. A pipeline is a high-level object that contains multiple steps. A function becomes a ZenML pipeline by being decorated with `@pipeline`, and a step when decorated with `@step`. This is a standard pattern when using orchestrators: you have a high-level function, often called a pipeline, that calls multiple units/steps/tasks.

Let's explore how we can implement a ZenML pipeline with one of the ML pipelines implemented for the LLM Twin project. In the code snippet below, we defined a ZenML pipeline that queries the database for a user based on its full name and crawls all the provided links under that user:

```
from zenml import pipeline
from steps.etl import crawl_links, get_or_create_user
@pipeline
def digital_data_etl(user_full_name: str, links: list[str]) -> None:
    user = get_or_create_user(user_full_name)
    crawl_links(user=user, links=links)
```

You can run the pipeline with the following CLI command: `poetry poe run-digital-data-etl`. To visualize the pipeline run, you can go to your ZenML dashboard (at `http://127.0.0.1:8237/`) and, on the left

panel, click on the **Pipelines** tab and then on the **digital_data_etl** pipeline, as illustrated in *Figure 2.2*:

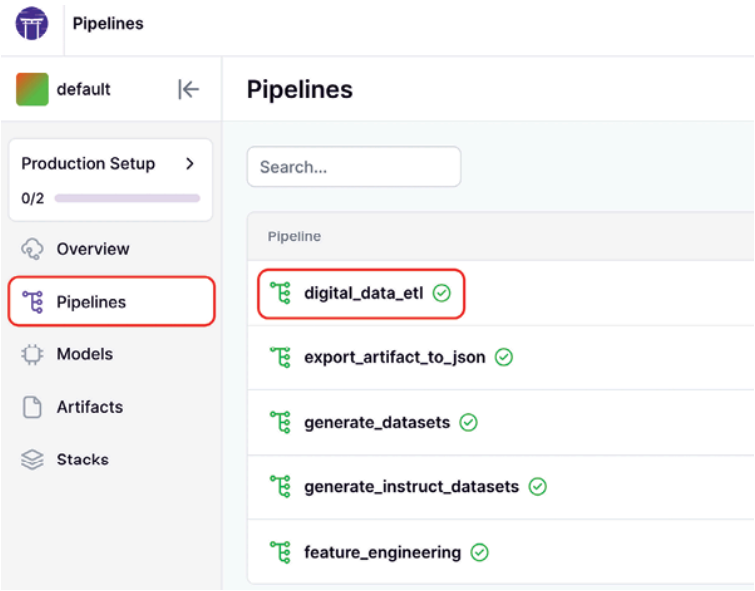

Figure 2.2: ZenML Pipelines dashboard

After clicking on the **digital_data_etl** pipeline, you can visualize all the previous and current pipeline runs, as seen in *Figure 2.3*. You can see which one succeeded, failed, or is still running. Also, you can see the stack used to run the pipeline, where the default stack is the one used to run your ML pipelines locally.

Figure 2.3: ZenML digital_data_etl pipeline dashboard. Example of a specific pipeline

🔍**Quick tip**: Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

🔒 **The next-gen Packt Reader** and a **free PDF/ePub copy** of this book are included with your purchase. Unlock them by scanning the QR code below or visiting https://www.packtpub.com/unlock/9781836200079.

Now, after clicking on the latest **digital_data_etl** pipeline run (or any other run that succeeded or is still running), we can visualize the pipeline's steps, outputs, and insights, as illustrated in *Figure 2.4*.

This structure is often called a **directed acyclic graph** (**DAG**). More on DAGs in *Chapter 11*.

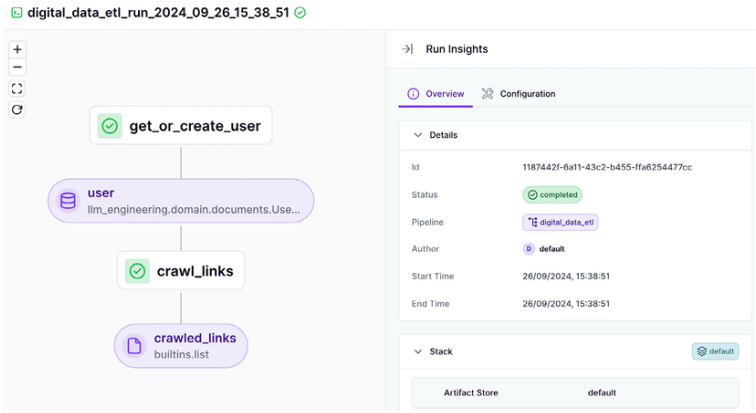

Figure 2.4: ZenML digital_data_etl pipeline run dashboard (example of a specific pipeline run)

By clicking on a specific step, you can get more insights into its code and configuration. It even aggregates the logs output by that specific step to avoid switching between tools, as shown in *Figure 2.5*.

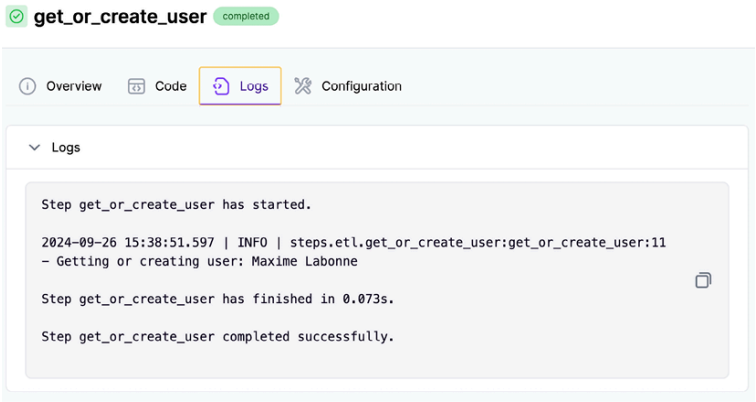

Figure 2.5: Example of insights from a specific step of the digital_data_etl pipeline run

Now that we understand how to define a ZenML pipeline and how to look it up in the dashboard, let's quickly look at how to define a ZenML step. In the code snippet below, we defined the `get_or_create_user()` step, which works just like a normal Python function but is decorated with `@step`. We won't go into the details of the logic, as we will cover the ETL logic in *Chapter 3*. For now, we will focus only on the ZenML functionality.

```
from loguru import logger
from typing_extensions import Annotated
from zenml import get_step_context, step
from llm_engineering.application import utils
from llm_engineering.domain.documents import UserDocument
@step
def get_or_create_user(user_full_name: str) -> Annotated[UserDocument, "user"]:
    logger.info(f"Getting or creating user: {user_full_name}")
    first_name, last_name = utils.split_user_full_name(user_full_name)
    user = UserDocument.get_or_create(first_name=first_name, last_name=last_name)
    return user
```

Within a ZenML step, you can define any Python logic your use case needs. In this simple example, we are just creating or retrieving a user,

but we could replace that code with anything, starting from data collection to feature engineering and training. What is essential to notice is that to integrate ZenML with your code, you have to write modular code, where each function does just one thing. The modularity of your code makes it easy to decorate your functions with `@step` and then glue multiple steps together within a main function decorated with `@pipeline`. One design choice that will impact your application is deciding the granularity of each step, as each will run as a different unit on a different machine when deployed in the cloud.

To decouple our code from ZenML, we encapsulated all the application and domain logic into the `llm_engineering` Python module. We also defined the `pipelines` and `steps` folders, where we defined our ZenML logic.

Within the `steps` module, we only used what we needed from the `llm_engineering` Python module (similar to how you use a Python package). In the `pipelines` module, we only aggregated ZenML steps to glue them into the final pipeline. Using this design, we can easily swap ZenML with another orchestrator or use our application logic in other use cases, such as a REST API. We only have to replace the ZenML code without touching the `llm_engineering` module where all our logic resides. This folder structure is reflected at the root of the LLM-Engineers-Handbook repository, as illustrated in *Figure 2.6*:

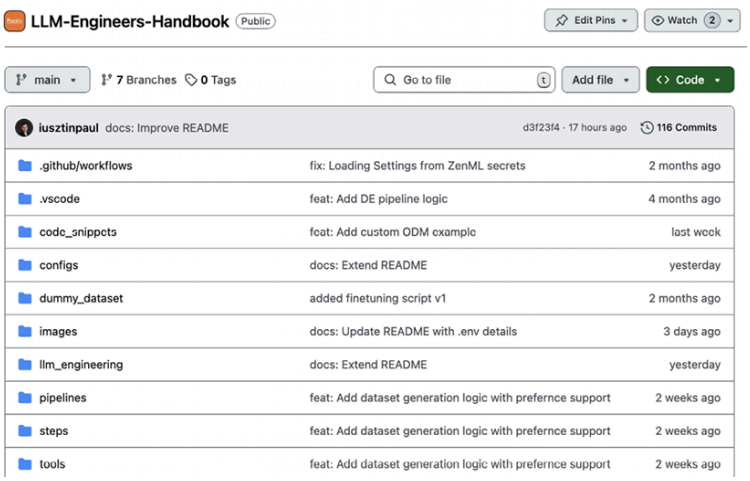

Figure 2.6: LLM-Engineers-Handbook repository folder structure

One last thing to consider when writing ZenML steps is that if you return a value, it should be serializable. ZenML can serialize most objects that can be reduced to primitive data types, but there are a few exceptions. For example, we used UUID types as IDs throughout the code, which aren't natively supported by ZenML. Thus, we had to extend ZenML's materializer to support UUIDs. We raised this issue to ZenML. Hence, in future ZenML versions, UUIDs will be supported, but it was an excellent example of the serialization aspect of transforming function outputs in artifacts.

Artifacts and metadata

As mentioned in the previous section, ZenML transforms any step output into an artifact. First, let's quickly understand what an artifact is. In MLOps, an **artifact** is any file(s) produced during the machine learning

lifecycle, such as datasets, trained models, checkpoints, or logs. Artifacts are crucial for reproducing experiments and deploying models. We can transform anything into an artifact. For example, the model registry is a particular use case for an artifact. Thus, artifacts have these unique properties: they are versioned, sharable, and have metadata attached to them to understand what's inside quickly. For example, when wrapping your dataset with an artifact, you can add to its metadata the size of the dataset, the train-test split ratio, the size, types of labels, and anything else useful to understand what's inside the dataset without actually downloading it.

Let's circle back to our **digital_data_etl** pipeline example, where we had as a step output an artifact, the crawled links, which are an artifact, as seen in *Figure 2.7*

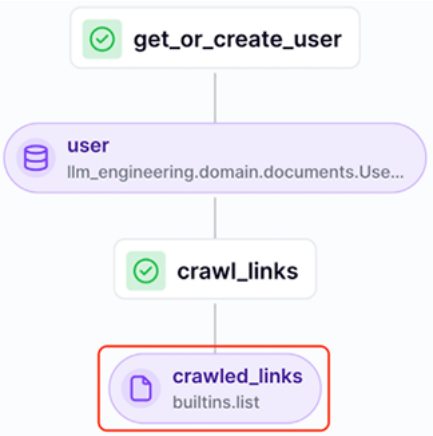

Figure 2.7: ZenML artifact example using the digital_data_etl pipeline as an example

By clicking on the `crawled_links` artifact and navigating to the **Metadata** tab, we can quickly see all the domains we crawled for a particular author, the number of links we crawled for each domain, and how many were successful, as illustrated in *Figure 2.8*:

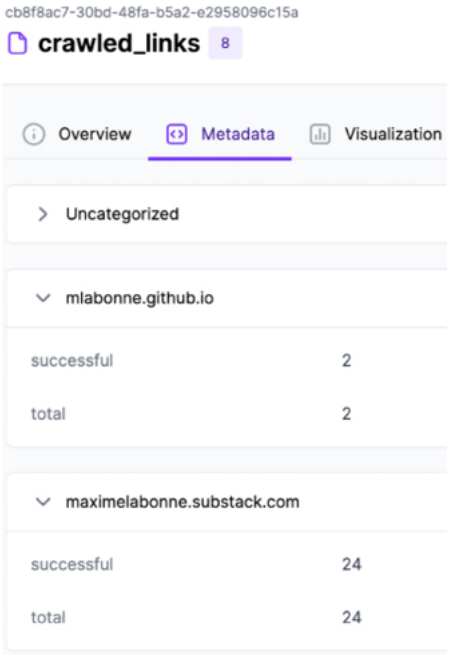

Figure 2.8: ZenML metadata example using the digital_data_etl pipeline as an example

A more interesting example of an artifact and its metadata is the generated dataset artifact. In *Figure 2.9*, we can visualize the metadata of the `instruct_datasets` artifact, which was automatically generated and will be used to fine-tune the LLM Twin model. More details on the `instruction datasets` are in *Chapter 5*. For now, we want to highlight that within the dataset's metadata, we have precomputed a lot of helpful information about it, such as how many data categories it contains, its storage size, and the number of samples per training and testing split.

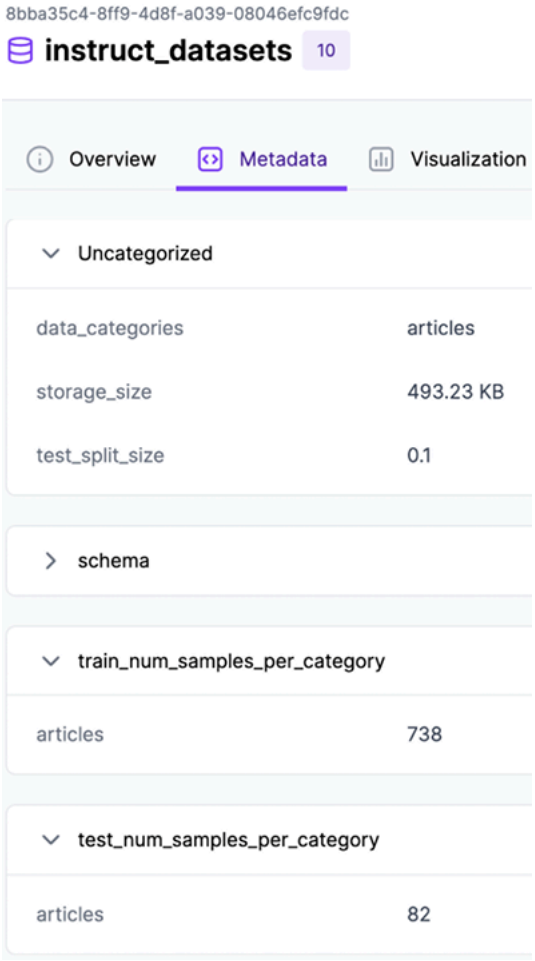

Figure 2.9: ZenML metadata example for the instruct_datasets artifact

The metadata is manually added to the artifact, as shown in the code snippet below. Thus, you can precompute and attach to the artifact's metadata anything you consider helpful for dataset discovery across your business and projects:

```
… # More imports
from zenml import ArtifactConfig, get_step_context, step
@step
def generate_intruction_dataset(
    prompts: Annotated[dict[DataCategory, list[GenerateDatasetSamplesPrompt]], "pr
    InstructTrainTestSplit,
    ArtifactConfig(
        name="instruct_datasets",
        tags=["dataset", "instruct", "cleaned"],
    ),
```

```
]:
    datasets = … # Generate datasets
    step_context = get_step_context()
    step_context.add_output_metadata(output_name="instruct_datasets", metadata=_ge
    return datasets
def _get_metadata_instruct_dataset(datasets: InstructTrainTestSplit) -> dict[str,
    instruct_dataset_categories = list(datasets.train.keys())
    train_num_samples = {
        category: instruct_dataset.num_samples for category, instruct_dataset in da
    }
    test_num_samples = {category: instruct_dataset.num_samples for category, instru
    return {
        "data_categories": instruct_dataset_categories,
        "test_split_size": datasets.test_split_size,
        "train_num_samples_per_category": train_num_samples,
        "test_num_samples_per_category": test_num_samples,
    }
```

Also, you can easily download and access a specific version of the dataset using its **Universally Unique Identifier** (**UUID**), which you can find using the ZenML dashboard or CLI:

```
from zenml.client import Client
artifact = Client().get_artifact_version('8bba35c4-8ff9-4d8f-a039-08046efc9fdc')
loaded_artifact = artifact.load()
```

The last step in exploring ZenML is understanding how to run and configure a ZenML pipeline.

How to run and configure a ZenML pipeline

All the ZenML pipelines can be called from the `run.py` file, accessed at `tools/run.py` in our GitHub repository. Within the `run.py` file, we implemented a simple CLI that allows you to specify what pipeline to run. For example, to call the `digital_data_etl` pipeline to crawl Maxime's content, you have to run:

```
python -m tools.run --run-etl --no-cache --etl-config-filename digital_data_etl_ma
```

Or, to crawl Paul's content, you can run:

```
python -m tools.run --run-etl --no-cache --etl-config-filename digital_data_etl_pa
```

As explained when introducing Poe the Poet, all our CLI commands used to interact with the project will be executed through Poe to simplify and standardize the project. Thus, we encapsulated these Python calls under the following `poe` CLI commands:

```
poetry poe run-digital-data-etl-maxime
poetry poe run-digital-data-etl-paul
```

We only change the ETL config file name when scraping content for different people. ZenML allows us to inject specific configuration files at runtime as follows:

```
config_path = root_dir / "configs" / etl_config_filename
assert config_path.exists(), f"Config file not found: { config_path }"
run_args_etl = {
        "config_path": config_path,
        "run_name": f"digital_data_etl_run_{dt.now().strftime('%Y_%m_%d_%H_%M_%S')}
}
 digital_data_etl.with_options()(**run_args_etl)
```

In the config file, we specify all the parameters that will input the pipeline as parameters. For example, the `configs/digital_data_etl_maxime_labonne.yaml` configuration file looks as follows:

```
parameters:
  user_full_name: Maxime Labonne # [First Name(s)] [Last Name]
  links:
    # Personal Blog
    - https://mlabonne.github.io/blog/posts/2024-07-29_Finetune_Llama31.html
    - https://mlabonne.github.io/blog/posts/2024-07-15_The_Rise_of_Agentic_Data_Ge
    # Substack
    - https://maximelabonne.substack.com/p/uncensor-any-llm-with-abliteration-d301
    … # More links
```

Where the `digital_data_etl` function signature looks like this:

```
@pipeline
def digital_data_etl(user_full_name: str, links: list[str]) -> str:
```

This approach allows us to configure each pipeline at runtime without modifying the code. We can also clearly track the inputs for all our pipelines, ensuring reproducibility. As seen in *Figure 2.10*, we have one or more configs for each pipeline.

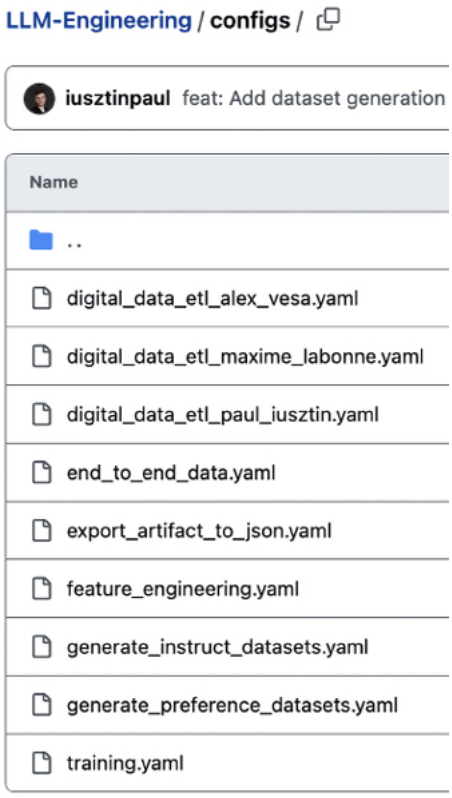

Figure 2.10: ZenML pipeline configs

Other popular orchestrators similar to ZenML that we've personally tested and consider powerful are Airflow, Prefect, Metaflow, and Dagster. Also, if you are a heavy user of Kubernetes, you can opt for Agro Workflows or Kubeflow, the latter of which works only on top of Kubernetes. We still consider ZenML the best trade-off between ease of use, features, and costs. Also, none of these tools offer the stack feature that is offered by ZenML, which allows it to avoid vendor-locking you in to any cloud ecosystem.

In *Chapter 11*, we will explore in more depth how to leverage an orchestrator to implement MLOps best practices. But now that we understand ZenML, what it is helpful for, and how to use it, let's move on to the experiment tracker.

Comet ML: experiment tracker

Training ML models is an entirely iterative and experimental process. Unlike traditional software development, it involves running multiple parallel experiments, comparing them based on predefined metrics, and deciding which one should advance to production. An experiment tracking tool allows you to log all the necessary information, such as metrics and visual representations of your model predictions, to compare all your experiments and quickly select the best model. Our LLM project is no exception.

As illustrated in *Figure 2.11*, we used Comet to track metrics such as training and evaluation loss or the value of the gradient norm across all our experiments.

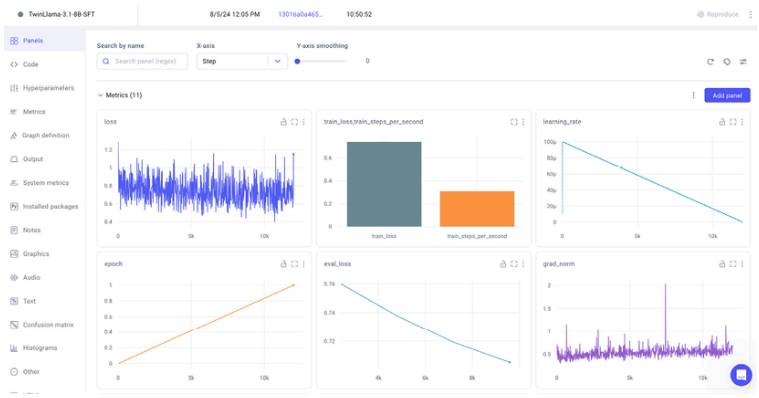

Figure 2.11: Comet ML training metrics example

Using an experiment tracker, you can go beyond training and evaluation metrics and log your training hyperparameters to track different configurations between experiments.

It also logs out-of-the-box system metrics such as GPU, CPU, or memory utilization to give you a clear picture of what resources you need during training and where potential bottlenecks slow down your training, as seen in *Figure 2.12*.

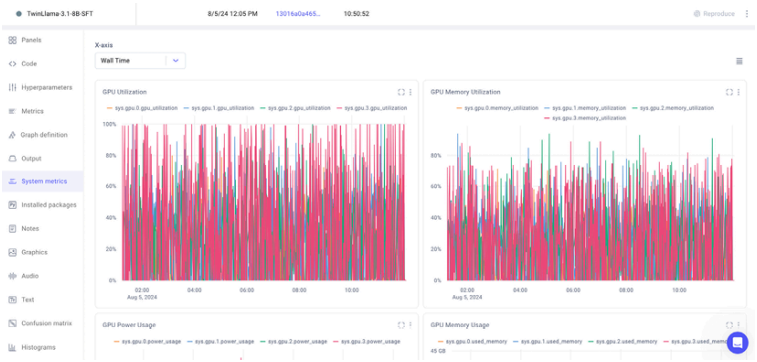

Figure 2.12: Comet ML system metrics example

You don't have to set up Comet locally. We will use their online version for free without any constraints throughout this book. Also, if you want to look more in-depth into the Comet ML experiment tracker, we made the training experiments tracked with Comet ML public while fine-tuning our LLM Twin models. You can access them here: https://www.comet.com/mlabonne/llm-twin-training/view/new/panels.

Other popular experiment trackers are W&B, MLflow, and Neptune. We've worked with all of them and can state that they all have mostly the same features, but Comet ML differentiates itself through its ease of use and intuitive interface. Let's move on to the final piece of the MLOps puzzle: Opik for prompt monitoring.

Opik: prompt monitoring

You cannot use standard tools and techniques when logging and monitoring prompts. The reason for this is complicated. We will dig into it in *Chapter 11*. However, to quickly give you some understanding, you cannot use standard logging tools as prompts are complex and unstructured chains.

When interacting with an LLM application, you chain multiple input prompts and the generated output into a trace, where one prompt depends on previous prompts.

Thus, instead of plain text logs, you need an intuitive way to group these traces into a specialized dashboard that makes debugging and monitoring traces of prompts easier.

We used Opik, an open-source tool made by Comet, as our prompt monitoring tool because it follows Comet's philosophy of simplicity and ease of use, which is currently relatively rare in the LLM landscape. Other options offering similar features are Langfuse (open source, https://langfuse.com), Galileo (not open source, rungalileo.io), and LangSmith (not open source, https://www.langchain.com/langsmith), but we found their solutions more cumbersome to use and implement. Opik, along with its serverless option, also provides a free open-source version that you have complete control over. You can read more on Opik at https://github.com/comet-ml/opik.

Databases for storing unstructured and vector data

We also want to present the NoSQL and vector databases we will use within our examples. When working locally, they are already integrated through Docker. Thus, when running `poetry poe local-infrastructure-up`, as instructed a few sections above, local images of Docker for both databases will be pulled and run on your machine. Also, when deploying the project, we will show you how to use their serverless option and integrate it with the rest of the LLM Twin project.

MongoDB: NoSQL database

MongoDB is one of today's most popular, robust, fast, and feature-rich NoSQL databases. It integrates well with most cloud ecosystems, such as AWS, Google Cloud, Azure, and Databricks. Thus, using MongoDB as our NoSQL database was a no-brainer.

When we wrote this book, MongoDB was used by big players such as Novo Nordisk, Delivery Hero, Okta, and Volvo. This widespread adoption suggests that MongoDB will remain a leading NoSQL database for a long time.

We use MongoDB as a NoSQL database to store the raw data we collect from the internet before processing it and pushing it into the vector database. As we work with unstructured text data, the flexibility of the NoSQL database fits like a charm.

Qdrant: vector database

Qdrant (https://qdrant.tech/) is one of the most popular, robust, and feature-rich vector databases. We could have used almost any vector database for our small MVP, but we wanted to pick something light and likely to be used in the industry for many years to come.

We will use Qdrant to store the data from MongoDB after it's processed and transformed for GenAI usability.

Qdrant is used by big players such as X (formerly Twitter), Disney, Microsoft, Discord, and Johnson & Johnson. Thus, it is highly probable that Qdrant will remain in the vector database game for a long time.

While writing the book, other popular options were Milvus, Redis, Weaviate, Pinecone, Chroma, and pgvector (a PostgreSQL plugin for vector indexes). We found that Qdrant offers the best trade-off between RPS, latency, and index time, making it a solid choice for many generative AI applications.

Comparing all the vector databases in detail could be a chapter in itself. We don't want to do that here. Still, if curious, you can check the *Vector DB Comparison* resource from Superlinked at https://superlinked.com/vector-db-comparison, which compares all the top vector databases in terms of everything you can think about, from the license and release year to database features, embedding models, and frameworks supported.

Preparing for AWS

This last part of the chapter will focus on setting up an AWS account (if you don't already have one), an AWS access key, and the CLI. Also, we will look into what SageMaker is and why we use it.

We picked AWS as our cloud provider because it's the most popular out there and the cloud in which we (the writers) have the most experience. The reality is that other big cloud providers, such as GCP or Azure, offer similar services. Thus, depending on your specific application, there is always a trade-off between development time (in which you have the most experience), features, and costs. But for our MVP, AWS, it's the perfect option as it provides robust features for everything we need, such as S3 (object storage), ECR (container registry), and SageMaker (compute for training and inference).

Setting up an AWS account, an access key, and the CLI

As AWS could change its UI/UX, the best way to instruct you on how to create an AWS account is by redirecting you to their official tutorial: https://docs.aws.amazon.com/accounts/latest/reference/manage-acct-creating.html.

After successfully creating an AWS account, you can access the AWS console at http://console.aws.amazon.com. Select **Sign in using root user email** (found under the **Sign in** button), then enter your account's email address and password.

Next, we must generate access keys to access AWS programmatically. The best option to do so is first to create an IAM user with administrative access as described in this AWS official tutorial: https://docs.aws.amazon.com/streams/latest/dev/setting-up.html

For production accounts, it is best practice to grant permissions with a policy of least privilege, giving each user only the permissions they require to perform their role. However, to simplify the setup of our test account, we will use the `AdministratorAccess` managed policy, which gives our user full access, as explained in the tutorial above and illustrated in *Figure 2.13*.

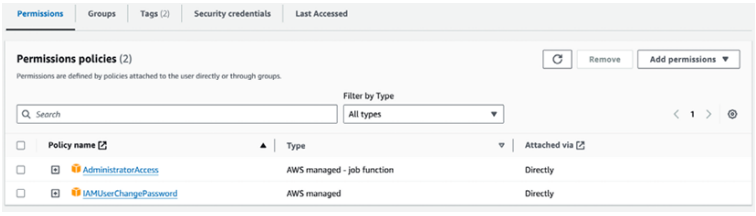

Figure 2.13: IAM user permission policies example

Next, you have to create an access key for the IAM user you just created using the following tutorial: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_access-keys.html.

The access keys will look as follows:

```
aws_access_key_id = <your_access_key_id>
aws_secret_access_key = <your_secret_access_key>
```

Just be careful to store them somewhere safe, as you won't be able to access them after you create them. Also, be cautious with who you share them, as they could be used to access your AWS account and manipulate various AWS resources.

The last step is to install the AWS CLI and configure it with your newly created access keys. You can install the AWS CLI using the following link: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html.

After installing the AWS CLI, you can configure it by running `aws configure`. Here is an example of our AWS configuration:

```
[default]
aws_access_key_id = *************
aws_secret_access_key = ************
region = eu-central-1
output = json
```

For more details on how to configure the AWS CLI, check out the following tutorial: https://docs.aws.amazon.com/cli/v1/userguide/cli-configure-files.html.

Also, to configure the project with your AWS credentials, you must fill in the following variables within your `.env` file:

```
AWS_REGION="eu-central-1" # Change it with your AWS region. By default, we use "eu-
AWS_ACCESS_KEY="<your_aws_access_key>"
AWS_SECRET_KEY="<your_aws_secret_key>"
```

An important note about costs associated with hands-on tasks in this book

All the cloud services used across the book stick to their freemium option, except AWS. Thus, if you use a personal AWS account, you will be responsible for AWS costs as you follow along in this book. While some services may fall under AWS Free Tier usage, others will not. Thus, you are responsible for checking your billing console regularly.

Most of the costs will come when testing SageMaker for training and inference. Based on our tests, the AWS costs can vary between $50 and $100 using the specifications provided in this book and repository.

See the AWS documentation on setting up billing alarms to monitor your costs at https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/monitor_estimated_charges_with_cloudwatch.html.

SageMaker: training and inference compute

The last topic of this chapter is understanding SageMaker and why we decided to use it. SageMaker is an ML platform used to train and deploy ML models. An official definition is as follows: AWS SageMaker is a fully managed machine learning service by AWS that enables developers and data scientists to build, train, and deploy machine learning models at scale. It simplifies the process by handling the underlying infrastructure, allowing users to focus on developing high-quality models efficiently.

We will use SageMaker to fine-tune and operationalize our training pipeline on clusters of GPUs and to deploy our custom LLM Twin model as a REST API that can be accessed in real time from anywhere in the world.

Why AWS SageMaker?

We must also discuss why we chose AWS SageMaker over simpler and more cost-effective options, such as AWS Bedrock. First, let's explain Bedrock and its benefits.

Amazon Bedrock is a serverless solution for deploying LLMs. Serverless means that there are no servers or infrastructure to manage. It provides pre-trained models, which you can access directly through API calls. When we wrote this book, they provided support only for Mistral, Flan, Llama 2, and Llama 3 (quite a limited list of options). You can send input data and receive predictions from the models without managing the underlying infrastructure or software. This approach significantly reduces the complexity and time required to integrate AI capabilities into applications, making it more accessible to developers with limited machine learning expertise. However, this ease of integration comes at the cost of limited customization options, as you're restricted to the pre-trained models and APIs provided by Amazon Bedrock. In terms of pricing, Bedrock uses a simple pricing model based on the number of API calls. This straightforward pricing structure makes it more efficient to estimate and control costs.

Meanwhile, SageMaker provides a comprehensive platform for building, training, and deploying machine learning models. It allows you to customize your ML processes entirely or even use the platform for research. That's why SageMaker is mainly used by data scientists and machine learning experts who know how to program, understand machine learning concepts, and are comfortable working with cloud platforms such as AWS. SageMaker is a double-edged sword regarding costs, following a pay-as-you-go pricing model similar to most AWS services. This means you have to pay for the usage of computing resources, storage, and any other services required to build your applications.

In contrast to Bedrock, even if the SageMaker endpoint is not used, you will still pay for the deployed resources on AWS, such as online EC2 instances. Thus, you have to design autoscaling systems that delete unused resources. To conclude, Bedrock offers an out-of-the-box solution that allows you to quickly deploy an API endpoint powered by one of the available foundation models. Meanwhile, SageMaker is a multi-functional platform enabling you to customize your ML logic fully.

So why did we choose SageMaker over Bedrock? Bedrock would have been an excellent solution for quickly prototyping something, but this is a

book on LLM engineering, and our goal is to dig into all the engineering aspects that Bedrock tries to mask away. Thus, we chose SageMaker because of its high level of customizability, allowing us to show you all the engineering required to deploy a model.

In reality, even SageMaker isn't fully customizable. If you want complete control over your deployment, use EKS, AWS's Kubernetes self-managed service. In this case, you have direct access to the virtual machines, allowing you to fully customize how you build your ML pipelines, how they interact, and how you manage your resources. You could do the same thing with AWS ECS, AWS's version of Kubernetes. Using EKS or ECS, you could also reduce the costs, as these services cost considerably less.

To conclude, SageMaker strikes a balance between complete control and customization and a fully managed service that hides all the engineering complexity behind the scenes. This balance ensures that you have the control you need while also benefiting from the managed service's convenience.

Summary

In this chapter, we reviewed the core tools used across the book. First, we understood how to install the correct version of Python that supports our repository. Then, we looked over how to create a virtual environment and install all the dependencies using Poetry. Finally, we understood how to use a task execution tool like Poe the Poet to aggregate all the commands required to run the application.

The next step was to review all the tools used to ensure MLOps best practices, such as a model registry to share our models, an experiment tracker to manage our training experiments, an orchestrator to manage all our ML pipelines and artifacts, and metadata to manage all our files and datasets. We also understood what type of databases we need to implement the LLM Twin use case. Finally, we explored the process of setting up an AWS account, generating an access key, and configuring the AWS CLI for programmatic access to the AWS cloud. We also gained a deep understanding of AWS SageMaker and the reasons behind choosing it to build our LLM Twin application.

In the next chapter, we will explore the implementation of the LLM Twin project by starting with the data collection ETL that scrapes posts, articles, and repositories from the internet and stores them in a data warehouse.

References

- Acsany, P. (2024, February 19). *Dependency Management With Python Poetry*. https://realpython.com/dependency-management-python-poetry/
- Comet.ml. (n.d.). *comet-ml/opik: Open-source end-to-end LLM Development Platform*. GitHub. https://github.com/comet-ml/opik
- Czakon, J. (2024, September 25). *ML Experiment Tracking: What It Is, Why It Matters, and How to Implement It*. neptune.ai. https://neptune.ai/blog/ml-experiment-tracking
- Hopsworks. (n.d.). *ML Artifacts (ML Assets)?* Hopsworks. https://www.hopsworks.ai/dictionary/ml-artifacts

- *Introduction | Documentation | Poetry – Python dependency management and packaging made easy*. (n.d.). https://python-poetry.org/docs
- Jones, L. (2024, March 21). *Managing Multiple Python Versions With pyenv*. https://realpython.com/intro-to-pyenv/
- Kaewsanmua, K. (2024, January 3). *Best Machine Learning Workflow and Pipeline Orchestration Tools*. neptune.ai. https://neptune.ai/blog/best-workflow-and-pipeline-orchestration-tools
- MongoDB. (n.d.). *What is NoSQL?* NoSQL databases explained. https://www.mongodb.com/resources/basics/databases/nosql-explained
- Nat-N. (n.d.). *nat-n/poethepoet: A task runner that works well with poetry*. GitHub. https://github.com/nat-n/poethepoet
- Oladele, S. (2024, August 29). *ML Model Registry: The Ultimate Guide*. neptune.ai. https://neptune.ai/blog/ml-model-registry
- Schwaber-Cohen, R. (n.d.). *What is a Vector Database & How Does it Work? Use Cases + Examples*. Pinecone. https://www.pinecone.io/learn/vector-database/
- *Starter guide | ZenML Documentation*. (n.d.). https://docs.zenml.io/user-guide/starter-guide
- *Vector DB Comparison*. (n.d.). https://superlinked.com/vector-db-comparison

Join our book's Discord space

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/llmeng

Answers collapsed